

Engenharia de Software

Universidade Federal de Uberlândia

Faculdade de Computação

Prof. Fabiano Dorça

DevOps

DevOps

- Logo, após um incremento resultante de um sprint — está pronto para entrar em produção.
 - Essa tarefa é conhecida pelos nomes de
 - implantação (deploy),
 - liberação (release) ou
 - entrega (delivery) do sistema.
 - Independentemente do nome, ela não é tão simples e rápida como pode parecer.

DevOps

- Historicamente, em organizações tradicionais, a **área de Tecnologia da Informação** era dividida em dois departamentos:
 - **Departamento de Sistemas** (ou Desenvolvimento), formado por desenvolvedores, programadores, analistas, arquitetos, etc.
 - **Departamento de Suporte** (ou Operações), no qual ficavam alocados os administradores de rede, administradores de bancos de dados, técnicos de suporte, técnicos de infraestrutura, etc.

DevOps

- Na maioria das vezes, a área de suporte tomava conhecimento de um sistema na véspera da sua implantação.
 - Consequentemente, a implantação poderia atrasar por meses, devido a uma variedade de problemas que não tinham sido identificados.
- Dentre eles, podemos citar
 - a falta de hardware para executar o novo sistema ou a nova funcionalidade,
 - problemas de desempenho,
 - incompatibilidades com o banco de dados de produção,
 - vulnerabilidades de segurança, etc.

DevOps

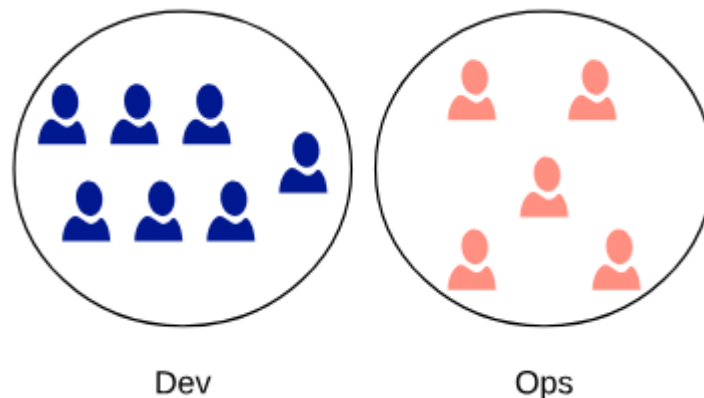
- No limite, esses problemas poderiam **resultar no cancelamento da implantação** e no abandono do sistema.
 - sysadmins — tomava conhecimento das características e **requisitos não-funcionais** de um novo software na véspera da implantação.
 - Esse problema era agravado pelo fato de os **sistemas serem monolitos**, cuja implantação gerava todo tipo de preocupação

DevOps

- Para facilitar a implantação e entrega de sistemas, foi proposto o conceito de DevOps.
- DevOps é então uma prática que visa unificar:
 - as culturas de desenvolvimento **(Dev)**
 - e de operação **(Ops)**,
 - visando **permitir a implantação mais rápida e ágil** de um sistema.

DevOps

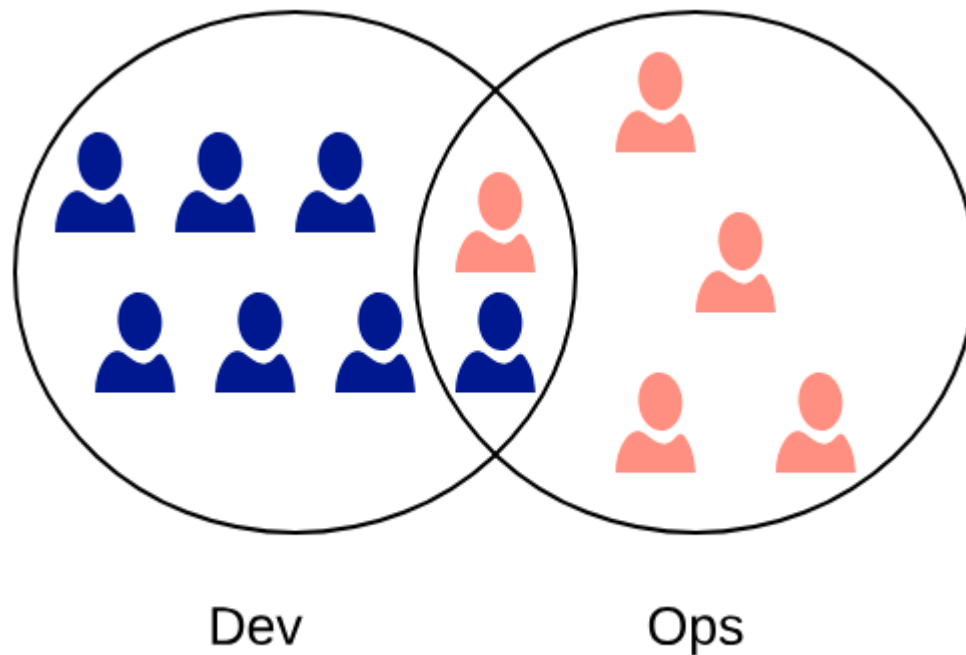
- Defende-se uma **aproximação entre o pessoal de desenvolvimento e o pessoal de operações** visando fazer com que a implantação de sistemas seja mais ágil e menos traumática.
 - a ideia é evitar dois silos independentes: desenvolvedores e operadores, com pouca ou nenhuma interação entre eles



VALENTE, Marco Tulio. Engenharia de software moderna. Princípios e práticas para desenvolvimento de software com produtividade, v. 1, n. 24, 2020.

DevOps

- Organização baseada em DevOps. Frequentemente, Devs e Ops sentam juntos para discutir questões sobre a entrega do sistema.



VALENTE, Marco Tulio. Engenharia de software moderna. Princípios e práticas para desenvolvimento de software com produtividade, v. 1, n. 24, 2020.

DevOps

- O termo **DevOps** começou a ser usado no final dos anos 2000 por profissionais frustrados com os **atritos constantes entre as equipes de desenvolvimento e de operações**.
- Então, eles convenceram-se de que uma solução seria a adoção de **princípios ágeis não apenas na fase de desenvolvimento, mas também na fase de implantação de sistemas**.
 - Para citar uma data precisa, em Novembro de 2009 foi realizada, na Bélgica, a primeira conferência da indústria sobre o tema, chamada DevOpsDays.

DevOps

- Quando migra-se para uma cultura de DevOps, os times ágeis podem incluir um profissional de operações, que participará dos trabalhos em tempo parcial ou mesmo em tempo integral.
 - Sempre em função da demanda, esse profissional pode também participar de mais de um time.
 - A ideia é que ele antecipe problemas de desempenho, segurança, incompatibilidades com outros sistemas, etc.
 - Ele pode também, enquanto o código está sendo implementado, começar a trabalhar nos scripts de instalação, administração e monitoramento do sistema em produção.

DevOps

- Desta forma, a **cultura DevOps melhora a agilidade do processo**, já que contribui para melhoria da comunicação entre as equipes Dev e Ops.
- Neste contexto, as **entregas (deploy)** ficam menos sujeitas a faltas em relação a
 - operação, infraestrutura, recursos de software e hardware, etc.
 - que podem prejudicar ou **atrasar as entregas**,
 - contribuindo então para a **cultura de agilidade** no que tange principalmente a entrega contínua.

DevOps

- DevOps advoga a automatização de todos os passos necessários para colocar um sistema em produção e monitorar o seu correto funcionamento.
- Isso requer a adoção de práticas, *notadamente testes automatizados*.
- Mas também requer o emprego de novas práticas e ferramentas, tais como
 - Integração Contínua (*Continuous Integration*) e
 - Deployment Contínuo (*Continuous Deployment*)

DevOps

- *Princípios para entrega de software:*
 - *Crie um processo repetível e confiável para entrega de software. Esse princípio é o mais importante deles.*
 - A ideia é que a entrega de software não pode ser um evento traumático, com passos manuais e sujeitos a surpresas.
 - Em vez disso, colocar um software em produção deve ser tão simples como apertar um botão.

DevOps

- *Automatize tudo que for possível.*
 - Na verdade, esse princípio é um pré-requisito do princípio anterior.
 - Advoga-se que todos os passos para entrega de um software devem ser automáticos, incluindo seu build, a execução dos testes, a configuração e ativação dos servidores e da rede, a carga do banco de dados, etc.
 - De novo, idealmente, queremos apertar um botão e, em seguida, ver o sistema em produção.

DevOps

- *Mantenha tudo em um sistema de controle de versões.*
 - Tudo no enunciado do princípio refere-se não apenas a todo o código fonte, mas também arquivos e scripts de administração do sistema, documentação, páginas Web, arquivos de dados, etc.
 - Consequentemente, deve ser simples restaurar e voltar o sistema para um estado anterior.

DevOps

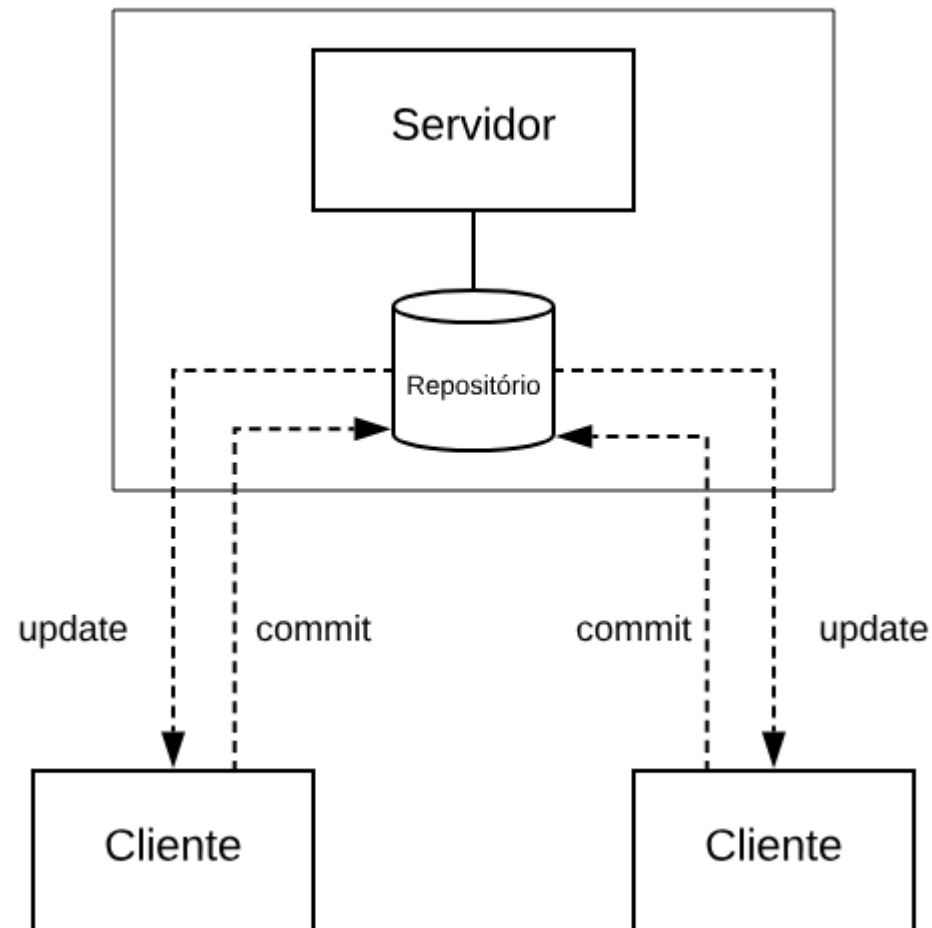
- **Controle de Versões (Version Control System)**
 - Um **Sistema de Controle de Versões (VCS**, na sigla em inglês) oferece os dois serviços mencionados no parágrafo anterior.
 - Primeiro, ele **oferece um repositório para armazenar a versão mais recente do código fonte de um sistema**, bem como de arquivos relacionados, como arquivos de documentação, configuração, páginas Web, manuais, etc.
 - Em segundo lugar, **ele permite que se recupere versões mais antigas de qualquer arquivo**, caso seja necessário.
 - Modernamente é inconcebível desenvolver qualquer sistema, mesmo que simples, sem um VCS.

DevOps

- A existência desse servidor é fundamental para que esses desenvolvedores possam colaborar e para que os operadores saibam precisamente qual versão do sistema deve ser colocada em produção.
- Além disso, sempre é útil manter o histórico das versões mais importantes de cada arquivo.
 - Isso permite, se necessário, realizar uma espécie de undo no tempo, isto é, recuperar o código de um arquivo como ele estava há anos atrás, por exemplo.

DevOps

- VCS Centralizado (início). Existe um único repositório, no nodo servidor.



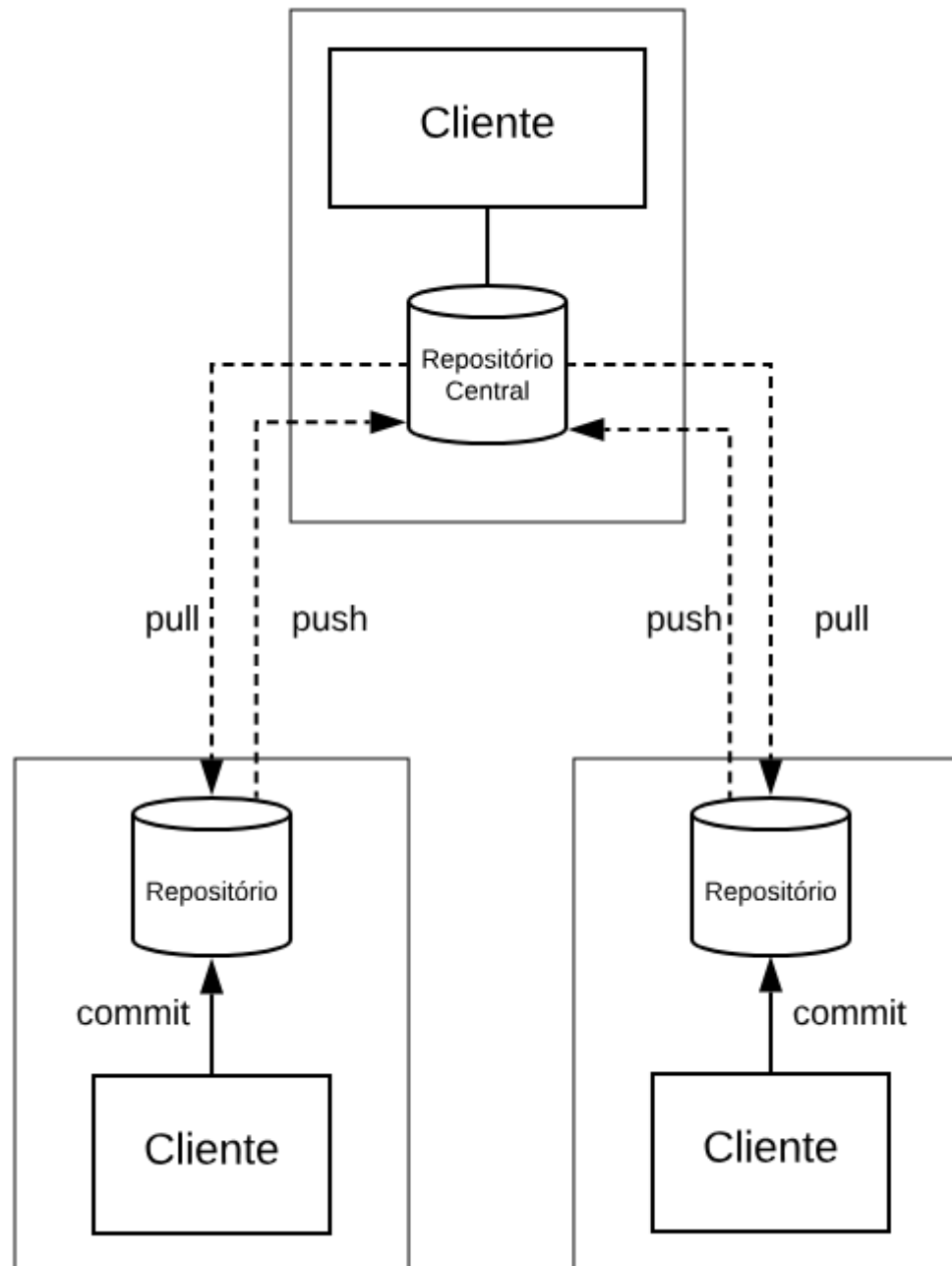
DevOps

- Os clientes acessam esse servidor para obter a versão mais recente de um arquivo.
- Feito isso, eles podem modificar o arquivo, por exemplo, para corrigir um bug ou implementar uma nova funcionalidade.
- Por fim, eles atualizam o arquivo no servidor, realizando uma operação chamada **commit**, que torna o arquivo visível para outros desenvolvedores.

DevOps

- No início dos anos 2000, começaram a **surgir Sistemas de Controle de Versões Distribuídos (DVCS)**.
- Dentre eles, podemos citar o sistema BitKeeper, cujo primeiro release é de 2000, e os sistemas Mercurial e **git**, ambos lançados em 2005
- Os DVCS contêm vários repositórios. Cada usuário tem seu próprio repositório e suas cópias de trabalho.

- VCS Distribuído (DVCS). Cada cliente possui um servidor.



DevOps

- Apenas confirmar suas alterações não dará a outras pessoas acesso às suas alterações.
- Isso ocorre porque o “commit” refletirá essas alterações em seu repositório local e você precisa enviá-las por “push” para torná-las visíveis no repositório central.

DevOps

- Cada desenvolvedor pode trabalhar de forma independente e até mesmo offline em sua máquina cliente, realizando commits no seu repositório.
- De tempos em tempos, ele deve sincronizar esse repositório com o central, por meio de duas operações: pull e push.
- Um **pull atualiza o repositório local com novos commits disponíveis no repositório central.**
- Por sua vez, um push faz a operação contrária, isto é, ele **envia para o repositório central os commits mais recentes realizados pelo desenvolvedor em seu repositório local.**

DevOps

- **Git é um sistema de controle de versões distribuído cujo desenvolvimento foi liderado por Linus Torvalds**, também responsável pela criação do sistema operacional Linux.
 - O Git é também um sistema de linha de comando.
 - Porém, existem clientes com interfaces gráficas, desenvolvidos por terceiros, que permitem usar o sistema sem ter que digitar comandos.
 - **GitHub é um serviço de hospedagem de código** que usa o sistema Git para prover controle de versões.
 - O **GitHub oferece repositórios públicos e gratuitos**, para projetos de código aberto, e repositórios fechados e pagos, para uso por empresas.

DevOps

- Assim, em vez de manter internamente um DVCS, uma empresa desenvolvedora de software pode alugar esse serviço do GitHub.
- Apesar do GitHub ser o mais popular, existem serviços semelhantes providos por outras empresas, como **GitLab e BitBucket**.
- Uma das principais funcionalidades que difere uma plataforma da outra é o ***foco que o GitLab vem dando à integração com ferramentas de DevOps***.
- O **GitLab proporciona, nativamente, ferramentas de integração e entrega contínua ou CI/CD**, além de métricas para acompanhamento de qualidade de código, performance e teste de usabilidade.

DevOps

- O que é Integração Contínua?
- **Integração Contínua (Continuous Integration ou CI)** é uma prática de desenvolvimento proposta por Extreme Programming (XP).
- O princípio motivador da prática: se uma tarefa causa dor, não podemos deixar que ela acumule.
 - Em vez disso, devemos quebrá-la em subtarefas que possam ser realizadas de forma frequente.
 - Como essas subtarefas são pequenas e simples, a dor decorrente da sua realização será menor (**babysteps**).

DevOps

- Sabemos que **grandes integrações são uma fonte de dor para os desenvolvedores**, pois eles têm que resolver de forma manual diversos conflitos.
 - Assim, CI recomenda integrar o código de forma frequente, isto é, contínua.
 - Com isso, as integrações serão pequenas e irão gerar menos **conflitos**.

DevOps

- Kent Beck, em seu livro de XP, defende o uso de CI da seguinte forma:

“Você deve integrar e testar o seu código em intervalos menores do que algumas horas. Programação em times não é um problema do tipo dividir-e-conquistar. Na verdade, é um problema que requer dividir, conquistar e integrar. A duração de uma tarefa de integração é imprevisível e pode facilmente levar mais tempo do que a tarefa original de codificação. Assim, quanto mais tempo você demorar para integrar, maiores e mais imprevisíveis serão os custos.”

DevOps

- Nessa citação, Beck defende várias integrações ao longo de um dia de trabalho de um desenvolvedor.
 - No entanto, essa recomendação não é consensual.
- Outros autores, como Martin Fowler, mencionam pelo menos uma integração por dia por desenvolvedor, o que parece ser um limite mínimo para um time argumentar que está usando CI.

DevOps

- **Boas Práticas para Uso de CI**
 - Quando usa-se CI, o sistema é constantemente atualizado com código novo.
 - ‘Para garantir que ele não seja quebrado — isto é, deixe de compilar ou possua bugs —, recomenda-se o uso de algumas práticas em conjunto com CI.

DevOps

- **Build Automatizado**

- Build é o nome usado para designar a compilação de todos os arquivos de um sistema, até a geração de uma versão executável.
- Quando se usa CI, o build deve ser automatizado, isto é, não incluir nenhum passo manual.
- Além disso, é importante que ele seja o mais rápido possível, pois com integração contínua ele será sempre executado.
- Alguns autores, por exemplo, chegam a recomendar um limite de 10 minutos para execução de um build.

DevOps

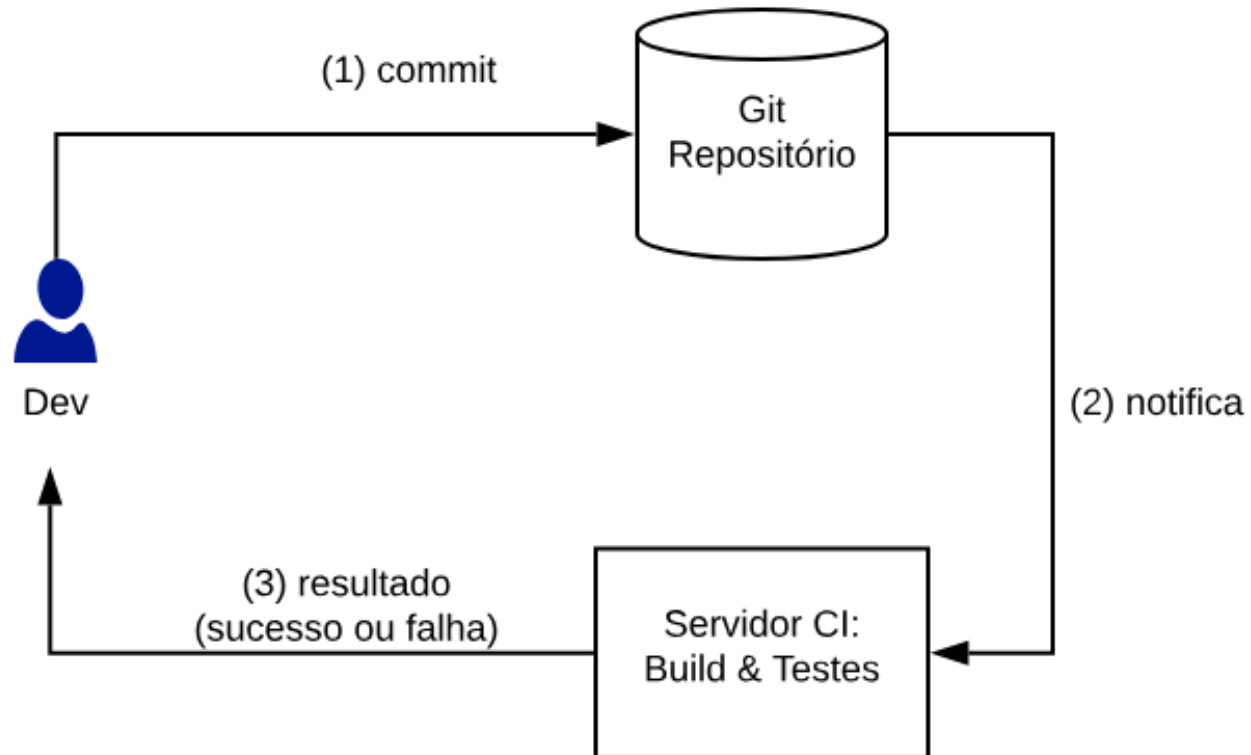
- **Testes Automatizados**

- Além de garantir que o sistema compila sem erros após cada nova integração, é importante garantir também que ele continua com o comportamento esperado.
- Por isso, ao usar CI, deve-se ter uma boa cobertura de testes, principalmente testes de unidade.

DevOps

- **Servidores de Integração Contínua**
- Por fim, os builds e testes automatizados devem ser executados com frequência, **se possível após cada novo commit realizado.**
- Para isso, existem **Servidores de CI**, que funcionam da seguinte forma:
 - Após um novo **commit**, o sistema de controle de versões avisa o servidor de CI, que clona o repositório e executa um build completo do sistema, bem como roda todos os testes.
 - Após a execução do build e dos testes, **o servidor notifica o desenvolvedor.**

DevOps



DevOps

- Se o servidor de CI notificar o desenvolvedor de que seu código não passou nos testes ou quebrou o build, **ele deve parar tudo o que está fazendo e providenciar a correção.**
- Isso é importante porque um **build quebrado impacta o trabalho dos outros desenvolvedores**, pois eles não vão conseguir mais compilar ou executar o sistema.
- Costuma-se dizer que nada em uma empresa de software tem **maior prioridade do que a correção de um build quebrado.**

DevOps

- No entanto, a solução pode ser simplesmente reverter o código para a versão anterior ao commit com problemas.
- Ainda nesta linha de raciocínio, um desenvolvedor somente deve avançar para uma próxima tarefa de programação após receber o resultado do servidor de CI.
- Por exemplo, ele não deve começar a escrever código novo antes de ter certeza de que seu último commit passou pelo serviço de CI.

DevOps

- Ele também não deve iniciar outras tarefas importantes, como entrar em uma reunião, sair para almoçar ou ir para a casa, antes do resultado desse servidor.
- Existe mais um passo da cadeia de automação proposta por DevOps, chamado **de Deployment Contínuo (Continuous Deployment ou CD)**.
- A diferença entre CI e CD é simples, mas seus impactos são profundos:
 - quando usa-se CD, **todo novo commit que chega entra rapidamente em produção**, em questões de horas, por exemplo.

DevOps

- O fluxo de trabalho quando se usa CD é o seguinte:
 - O desenvolvedor desenvolve e testa na sua máquina local.
 - Ele realiza um commit e o servidor de CI executa novamente um build e os testes de unidade.
 - Algumas vezes no dia, o servidor de CI realiza testes mais exaustivos com os novos commits que ainda não entraram em produção.
 - Esses testes incluem, por exemplo, testes de integração, testes de interface e testes de desempenho.
 - Se todos os testes passarem, os commits entram imediatamente em produção. E os usuários já vão interagir com a nova versão do código.

DevOps

- CD reduz o tempo de entrega de novas funcionalidades.
 - Por exemplo, suponha que as funcionalidades $F1, F2, \dots, F_n$ estão previstas para uma nova release de um sistema.
- No modo tradicional, todas elas devem ser implementadas e testadas, antes da liberação da nova release.
- Por outro lado, com CD, as funcionalidades são liberadas assim que ficam prontas.
 - Ou seja, **CD diminui o intervalo entre releases.**

DevOps

- Passa-se a ter mais releases, mas com um menor número de funcionalidades.
- CD **torna novas releases (ou implantações) um não-evento.**
Explicando melhor, não existe mais um dia D ou um deadline para entrega de novas releases.
- **Deadlines são uma fonte de stress para desenvolvedores e**
operadores de sistemas de software.
- A perda de um deadline, por exemplo, pode fazer com que uma funcionalidade somente entre em produção meses depois.

DevOps

- Além de reduzir o ***stress causado por deadlines, CD ajuda a manter os desenvolvedores motivados***, pois eles não ficam meses trabalhando sem receber feedback.
 - Em vez disso, os desenvolvedores rapidamente recebem retorno — vindo de usuários reais — sobre o sucesso ou não de suas tarefas.
 - Em linha com o item anterior, CD favorece experimentação e um estilo de desenvolvimento orientado por dados e feedback dos usuários.
- ***Novas funcionalidades entram rapidamente em produção.***
 - Com isso, recebe-se retorno dos usuários, que podem recomendar mudanças na implementação ou, no limite, o cancelamento de alguma funcionalidade.

DevOps

- Deployment Contínuo (CD) ***não é recomendável para certos tipos de sistemas, incluindo sistemas desktop*** (como uma IDE ou um navegador Web), aplicações móveis e aplicações embutidas em hardware.
 - Provavelmente, você não gostaria de ser notificado diariamente de que existe uma nova versão do navegador que usa em seu desktop, ou do sistema de rede social que usa em seu celular ou ainda de que um novo driver está disponível para sua impressora.
- Esses sistemas demandam um processo de instalação que não é transparente para seus usuários, como é a atualização de um sistema Web.

DevOps

- **Ferramentas de CI/CD (Integração Contínua e Entrega Contínua)**
 - Automatizam builds, testes e deploys.
 - Um pipeline de CI/CD (Integração Contínua/Entrega Contínua) automatiza o processo de desenvolvimento, desde a integração de código até a entrega em ambientes de produção.
 - Ele garante que as alterações de código sejam testadas e implantadas de forma rápida e confiável.

DevOps

- a) Pipelines de CI/CD
 - Jenkins:
 - Ferramenta open-source para orquestrar workflows.
 - GitHub Actions, GitLab CI/CD:
 - Integração nativa com repositórios Git.
 - CircleCI, Travis CI:
 - Alternativas em nuvem.

DevOps

- Atualmente, **o Jenkins**
 - é uma das ferramentas mais populares e amplamente utilizadas para pipelines de CI/CD, especialmente devido à sua natureza de código aberto, flexibilidade e vasta gama de plugins.
 - **É um servidor de automação de código aberto que pode ser usado para criar pipelines de CI/CD de ponta a ponta,**
 - desde a construção e teste até a implantação em diversos ambientes.

DevOps

- O **Jenkins** é uma ferramenta open source escrita em Java.
- Possibilita a integração com praticamente todas as outras ferramentas DevOps existentes, além de oferecer mais de mil plugins para estender seus recursos e torná-lo mais específico para as necessidades de cada usuário.
- O Jenkins também pode ser instalado em qualquer sistema operacional que execute Java.

- Outras ferramentas populares incluem **Azure Pipelines**, **GitHub Actions**, **Bitbucket Pipelines** e **GitLab CI**.
 - **Azure Pipelines**: Parte do ecossistema do Microsoft Azure, oferece uma solução completa para **automação de pipelines CI/CD**, integrando-se bem com outros serviços Azure.
 - **GitHub Actions**: Permite automatizar fluxos de trabalho de desenvolvimento, incluindo construção, teste e implantação, diretamente no repositório GitHub
 - **GitLab CI/CD**: Oferece um sistema integrado de CI/CD para o GitLab, permitindo a automação de processos de desenvolvimento e entrega. O **GitLab** é uma das ferramentas de CI/CD mais usadas no cenário DevOps.
 - **Bitbucket Pipelines**: Integrado ao Bitbucket, permite automatizar o código, desde o teste até a produção, dentro do próprio repositório, de acordo com a Atlassian.

DevOps

- Git:
 - É a ferramenta de controle de versão subjacente.
- GitHub/GitLab:
 - São plataformas que hospedam repositórios Git e oferecem ferramentas de colaboração e gerenciamento de projetos.
- GitHub Actions/GitLab CI/CD:
 - São ferramentas para *automatizar pipelines de CI/CD.*
- A escolha entre GitHub e GitLab depende das necessidades e preferências específicas do seu projeto e equipe.

DevOps

- A escolha das ferramentas depende do stack tecnológico, orçamento e necessidades específicas (ex.: testes de segurança vs. desempenho).
- Um fluxo (pipeline de CI/CD) típico combinaria:
 - GitHub Actions (CI/CD) →
 - SonarQube (análise estática) →
 - JUnit 5 + Mockito (Testes Unitários) →
 - Selenium/Cypress (testes UI) →
 - JMeter (testes de carga) →
 - OWASP (teste de segurança) →
 - New Relic (monitoramento em produção).

DevOps

- CI/CD (Integração e Entrega Contínua)
 - GitHub Actions ou GitLab CI/CD:
 - Pipeline exemplo:
 1. Build com Maven/Gradle.
 2. Análise estática com SonarQube.
 3. Execução de testes unitários e de integração.
 4. Deploy em ambiente de teste.
 5. Testes E2E e de carga.
 6. Deploy em produção (com aprovação manual).
 - Docker + Kubernetes:
 - Empacotamento da aplicação em containers para escalabilidade.

DevOps

- Dúvidas?